# NANOZONE: Scalable, Efficient, and Secure Memory Protection on Arm CCA

Shiqi Liu, Yongpeng Gao, Mingyang Zhang, and Jie Wang

arXiv:2506.07034v2 [cs.CR] 24 Sep 2026

***Abstract*—Arm Confidential Compute Architecture (CCA) provides hardware primitives for confidential computing, but its standard CVM-based deployment model still isolates software only at the VM granularity. This leaves two critical gaps: the guest OS remains in the Trusted Computing Base, and applications lack any intra-process isolation boundary, exposing secrets to vulnerabilities such as Heartbleed. While intra-address-space isolation offers a theoretical solution, realizing it under an adversarial OS faces a fundamental scalability-efficiency trade-off: existing hardware primitives are either strictly limited in domain capacity or incur prohibitive privileged-switching latency.**

**To bridge this gap, we propose NANOZONE, a novel isolation architecture built on Arm CCA primitives. Rather than extending the realm-world software stack, NANOZONE protects normal-world processes and enforces fine-grained intra-process isolation from a minimal root-world monitor. NANOZONE unifies the speed of user-space permission switching with the capacity of physical address space isolation to achieve effectively unlimited domain scalability. To mask the latency of privileged transitions, it employs a locality-aware scheduling policy that maximizes execution residency within the fast user-level tier. To defeat an adversarial OS, NANOZONE anchors its root of trust in the root world, offloading critical isolation enforcement away from the untrusted kernel. Against intra-process adversaries, it further leverages hardware-assisted Code-Pointer Integrity (CPI) to prevent domain-switching abuse. We prototyped NANOZONE on Arm's official emulator and on physical hardware. Evaluation on real-world server applications shows that, compared to process-level isolation systems, our fine-grained protection preserves about 95% of their throughput.**

***Index Terms*—Arm CCA, Confidential computing, Trusted execution environments, Intra-process isolation.**

## I. INTRODUCTION

Confidential Computing protects tenant assets using hardware-based Trusted Execution Environments (TEEs) such as AMD SEV [1] and Intel TDX [2]. In the Arm ecosystem, the recently introduced *Confidential Compute Architecture (CCA)* [3] adds a *realm world* to Armv9-A to host Confidential Virtual Machines (CVMs). However, this coarse-grained isolation model leaves CCA exposed to two critical vectors. (1) **The Threat of an Adversarial Kernel.** Current architectures implicitly trust the guest OS, forcing it into the Trusted Computing Base (TCB). With monolithic kernels (e.g., Linux) exceeding 40M lines of code (LoCs), this design creates a massive attack surface where privilege-escalation bugs [4], [5] allow an adversary to compromise the entire CVM. (2) **Unmitigated Intra-Process Risks.** While CCA shields the VM from the host, it lacks primitives for fine-grained protection within the application itself. Complex software like OpenSSL (900K+ LoCs) handles secrets in only a fraction of its code base. Without intra-address-space boundaries, a single memory safety bug like Heartbleed [6] can leak secrets from non-critical code paths.

In response to the threat of an adversarial kernel, prior research has focused on TCB reduction. Gramine-TDX [7] replaces the monolithic kernel with a leaner LibOS ($\sim$57K LoCs), while RContainer [8] minimizes the trusted computing base via a mini-OS ($<$5K LoCs) on the CCA platform. Shelter [9] goes further by completely removing the OS from the trust boundary, enforcing process-level isolation. While Shelter effectively shields the application from the kernel, it fails to mitigate the *internal attack surface* described above.

Intra-address-space isolation is not a new concept in TEEs. For instance, in the context of Intel SGX [10], Nested Enclave [11] partitions an enclave into outer and inner layers through hardware modifications. However, the switching time between these layers (1.1 µs, comparable to native ecall/ocall latency) makes it impractical for high-frequency domain transitions. To further optimize performance, LightEnclave [12] employs Memory Protection Keys (MPK) [13] to partition an enclave into multiple domains. The performance gain stems from complete user-space domain switches that require only about 20 cycles. However, applying these established concepts to Arm CCA presents unique challenges. The CVM-style isolation in CCA and enclave-style isolation in SGX differ substantially in their architectural abstractions and hardware primitives. Furthermore, although solutions like NestedSGX [14], Veil [15], and CAP-VMs [16] have demonstrated intra-CVM isolation, adapting them to the standard Armv9 architecture is impractical. These approaches rely on either experimental hardware extensions (e.g., CHERI capabilities [17] in CAP-VMs) or vendor-specific features (e.g., AMD VMPLs in Veil) that are absent in the Arm CCA specification. Consequently, porting them to Arm would incur prohibitive overheads from complex hypervisor emulation.

For multi-tenant servers that hold secrets for mutually distrusting clients within a single process, such as per-client TLS keys in Nginx, these limitations call for a deployable Arm CCA design that combines intra-process isolation with a root of trust outside the OS.

This paper introduces NANOZONE, a hardware-assisted

This work was supported in part by the National Natural Science Foundation of China under Grant 62202194 and in part by the CCF-Ant Research Fund. *(Corresponding author: Jie Wang.)*

Shiqi Liu, Yongpeng Gao, Mingyang Zhang, and Jie Wang are with Hubei Key Laboratory of Distributed System Security, Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan, 430074, China (e-mail: shiqiliu@hust.edu.cn; sternen_hust@hust.edu.cn; zoneshiyi@hust.edu.cn; wangjie_s@hust.edu.cn).

intra-process isolation system built on Arm CCA primitives. NANOZONE aims to provide fine-grained isolation under an adversarial OS while minimizing the trusted software stack. To this end, NANOZONE places protected processes in the normal world and anchors security enforcement in a minimal root-world monitor at EL3, rather than extending the Realm/CVM software stack and trusting the additional Realm Management Monitor (RMM, 46K LoCs) at Realm EL2.

Efficient fine-grained isolation also requires fast domain switching. Intel MPK is a promising model because it supports user-space permission changes without costly kernel traps or TLB flushes, but it is unavailable on Arm. Although Armv7 previously provided a similar feature through memory domains [18], [19], its Domain Access Control Register is inaccessible from user space, and the feature has been deprecated in AArch64. Fortunately, Arm recently introduced the *Permission Overlay Extension (POE)* [20] in Armv8.9-A, making efficient user-space domain switching feasible on Arm. NANOZONE partitions memory pages into isolation domains by encoding POE indices, analogous to MPK protection keys, in Page Table Entries (PTEs). On authorized domain switches, NANOZONE temporarily grants access to sensitive memory by updating POE permission registers in user space, keeping the access window short. Because POE permissions are maintained per core, other cores remain restricted by their own permission-register state.

While the high-level design of NANOZONE may seem straightforward, it faces challenges spanning scalability, efficiency, and security. The core contribution of this paper lies in balancing the trade-offs across these three dimensions.

**Scalability Challenge: Expanding Domains beyond Hardware Limits.** POE is strictly limited to 7 usable domains, which is insufficient for multi-tenant server workloads. While Arm CCA's GPT mechanism supports unlimited domains, relying solely on it necessitates frequent privileged interventions, which undermines the lightweight nature of user-space isolation [21]. The core challenge is to architect a mechanism that achieves infinite scalability within the rigid constraints of hardware resources. NANOZONE overcomes this by integrating complementary hardware features into a cohesive three-tier zone design (§IV-A). First, it combines POE with the *Permission Indirection Extension (PIE)* [20], mapping unused PIE indexes to existing POE indexes to expand user-level domains from 7 to 28. Second, to break the hard limit, it exploits *Granule Protection Tables (GPT)* [22] and *GPC3* bypass windows [23] to reuse these indexes across distinct Physical Address Spaces (PAS). This hierarchical approach enables it to scale to an *unlimited* number of domains.

**Efficiency Challenge: Amortizing the Cost of Hierarchical Isolation.** The hierarchical scalability established above introduces a fundamental trade-off: accessing deeper tiers (PIE or PAS) incurs prohibitively higher switching costs. Unlike lightweight POE switches, crossing these boundaries requires trapping into higher privilege levels, resulting in $\sim$80$\times$ higher latency. The challenge is to sustain high throughput when the underlying isolation mechanism is inherently expensive. NANOZONE addresses this by enforcing a tier-aware execution model that accounts for the heterogeneous costs of domain switching. We exploit request locality to mask the latency of privileged traps, strategically scheduling requests to maximize residency within the currently active low-latency POE layer. As shown in Figure 1, this approach maintains a 96.72% hit rate in the fastest tier, effectively restricting expensive transitions to the bare minimum. Consequently, NANOZONE amortizes the heavy hardware isolation costs, reducing the average domain-switch latency to near-native speeds (only 4.87% of a raw privileged switch).

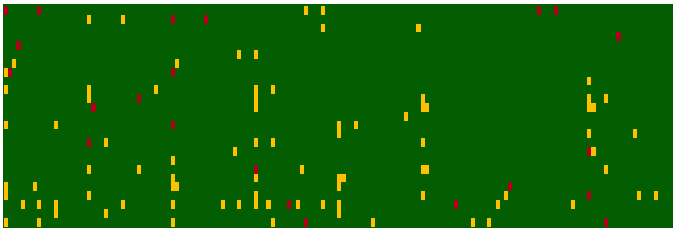


Fig. 1. Domain-switching scenarios for 1,000 Memcached requests in NANOZONE (green: POE switches; yellow: PIE switches; red: PAS switches).

**Security Challenge: Enforcing Integrity against an Adversarial Kernel.** Standard Control-Flow Integrity (CFI) relies on the critical premise that the OS enforces memory isolation to prevent code and pointer corruption. In our threat model, however, this assumption is invalidated. An adversarial OS can subvert isolation boundaries by manipulating page tables, rendering user-space CFI mechanisms ineffective. The challenge, therefore, is to restore the integrity of the isolation layer as a prerequisite for robust CFI. NANOZONE addresses this dependency by re-anchoring the root of trust in the monitor. First, to restore isolation, we incorporate a security module (§IV-C) that validates page table updates and protects sensitive registers, precluding the OS from remapping or tampering with domain memory. With isolation secured, we then enforce CFI by combining the *Guarded Control Stack (GCS)* [24] with a monitor-managed *Pointer-Integrity Memory (PIM)* (§IV-B), ensuring execution flow remains tamper-resistant even against kernel-level interference.

Since no commercial hardware yet supports the required Arm features, we built two prototypes: a functional version on Arm's official Fixed Virtual Platform and a performance version on a hardware SoC. Security evaluations confirm that NANOZONE effectively mitigates both intra-process vulnerabilities and privileged attacks via surface analysis (§VI-B) and CVE reproduction (§VI-C). Performance assessments on micro-benchmarks (§VII-B) and real-world applications (§VII-C) show that NANOZONE incurs only moderate overhead. For instance, while Nginx exhibits a 21.84% average throughput overhead, the marginal cost of our fine-grained protection remains small: compared to the state-of-the-art Shelter [9] (which lacks intra-process isolation), NANOZONE introduces only a 5.15% relative drop.

In summary, we make the following contributions:

- We propose a hierarchical model to resolve the *scalability-efficiency trade-off* in Arm hardware. It unifies the speed of POE with the scale of GPT, enabling unlimited isolation domains without sacrificing user-space

TABLE I
ACRONYMS AND KEY REGISTERS USED IN THIS PAPER.

| Abbrev. | Full Name | Abbrev. | Full Name | Register | Full Name |
|---|---|---|---|---|---|
| CCA | Confidential Compute Architecture | GPC | Granule Protection Check | POR_EL0 | POE permission register |
| RME | Realm Management Extension | GPC3 | Granule Protection Check 3 | PIRE0_EL1 | PIE permission register |
| CVM | Confidential Virtual Machine | GPF | Granule Protection Fault | GPCBW_EL3 | GPC bypass-window register |
| PAS | Physical Address Space | GCS | Guarded Control Stack | GCSPR_EL0 | GCS pointer register |
| GPT | Granule Protection Table | CPI | Code-Pointer Integrity | TPIDRRO_EL0 | Read-only thread pointer register |
| POE | Permission Overlay Extension | PIM | Pointer-Integrity Memory | | |
| PIE | Permission Indirection Extension | PTE | Page Table Entry | | |

performance.

- We redesign protection mechanisms for the threat model where the OS is adversarial. Anchoring trust in the monitor, NANOZONE enforces control-flow integrity (via GCS and CPI) independent of kernel-managed page tables.
- We show that our *locality-aware scheduling* effectively masks the latency of privileged transitions. Evaluation confirms that it retains about 95% of the throughput of process-level isolation, proving the practicality of fine-grained isolation.

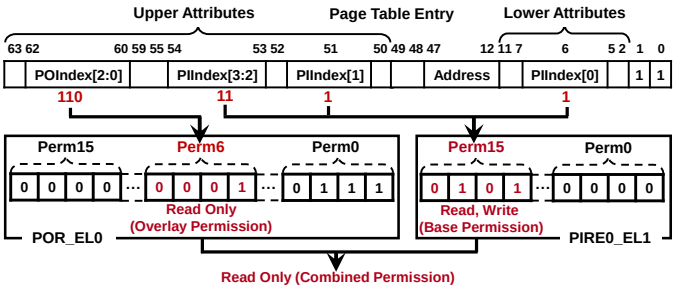



Fig. 2. Combination of permission indirection and overlay.

## II. BACKGROUND AND MOTIVATION

### A. Hardware Background

For ease of reference, Table I summarizes the main architectural acronyms and key registers used in the paper.

**Permission Indirection Extension (PIE).** Arm previously used a direct memory permission model, where permissions for each memory page were determined by the permission bits in the PTE. In this model, modifying the permissions of a domain requires updating the PTEs of all pages associated, making it inefficient for domain-based isolation. Additionally, the limited number of PTE bits restricts scalability for supporting new permissions. To address these limitations, Arm introduced an indirect permission model [20] in Armv8.9-A. This model manages user-space permissions using the 64-bit `PIRE0_EL1` register, where each 4 bits specify the permissions for a domain, supporting up to 16 domains. The original PTE permission bits (i.e., bit 54 `UXN`, bit 53 `PXN`, bit 51 `WRITE/DBM`, and bit 6 `USER/AP[1]`) are repurposed as indexes to indicate a page's domain. Notably, the direct and indirect permission models are compatible. As shown in Figure 2, in the direct model, a user-space memory page with read-write permissions has the bits `UXN` = 1, `PXN` = 1, `WRITE` = 1, and `USER` = 1 set. In the indirect model, these bits instead serve as an index in `PIRE0_EL1` (i.e., `perm15`), which is pre-configured with read-write permissions (i.e., `0b0101`). Among the 16 domains supported by `PIRE0_EL1`, 12 are pre-configured with fixed permissions to cover common use cases.

**Permission Overlay Extension (POE).** Although PIE can configure base permissions, permission switching requires trapping to privileged mode and flushing the TLB, leading to inefficiencies. To address this, Arm also introduced the POE [20] in Armv8.9-A, delegating part of permission management to user-space. As shown in Figure 2, base permissions can be further restricted by overlay permissions stored in the 64-bit `POR_EL0` register, which do not need to be cached in the TLB. During runtime, the MMU transparently combines and checks both base and overlay permissions. POE uses three unused bits [62:60] in the PTE as the index, supporting up to 8 POE domains[1]. However, the POE index defaults to 0, where `perm0` is fixed with full permissions (i.e., `0b0111`, read, write, and execute) to ensure that default base permissions are not overridden. As a result, only 7 POE domains are available for general use.

**Guarded Control Stack (GCS).** Arm introduced GCS [24] in Armv9.4-A to defend against ROP [25], similar to the x86 Shadow Stack [26]. GCS leverages a new permission (i.e., `perm11` in `PIRE0_EL1`) introduced by PIE, allowing non-privileged read operations while disallowing writes, except through the specific `GCSSTR` instruction. Notably, POE does not apply to GCS. When enabled, GCS allocates a protected stack for each thread in PIE `domain11`, pointed to by the GCS pointer register (`GCSPR_EL0`). During branch instructions with a link (e.g., `BL`, `BLR`), the CPU saves the return address in the link register (`LR`) and transparently pushes it onto the protected stack. Similarly, during a `RET` instruction, the CPU verifies the `LR` against the top of the protected stack to ensure return address integrity.

**Confidential Compute Architecture (CCA).** In the Armv9.2-A, Arm introduced the Confidential Compute Architecture (CCA) [3] with a hardware primitive called the Realm Management Extension (RME) [22]. CCA introduces a new execution environment called the realm world and separates the entire Exception Level 3 (EL3) into a distinct root world. Confidential Virtual Machines (CVMs), originally created by the normal-world hypervisor, are executed in the realm world,

[1]In Armv9.4-A, the 128-bit translation tables [24] extend support for a 4-bit POE index, enabling all 16 POE domains. However, software stack support for the 128-bit translation tables is still under development.

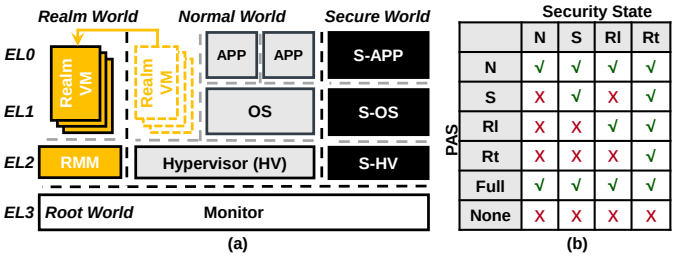



Fig. 3. The architecture of CCA and its PAS access permissions.

as shown in Figure 3(a). RME achieves fine-grained physical memory access control by leveraging two main steps: configuring Physical Address Spaces (PAS) through the Granule Protection Table (GPT) and enforcing PAS access checks with the Granule Protection Check (GPC). Specifically, the GPT maps each physical memory page (at a 4KB granularity) to one of several PAS: `normal`, `secure`, `realm`, `root`, `full-access`, or `no-access`. During memory accesses, the MMU consults the GPT to retrieve PAS of the target memory page and performs a GPC to validate access permissions based on the current security state, as illustrated in Figure 3(b). If an access is invalid, a Granule Protection Fault (GPF) is triggered immediately. The most privileged root world ensures that other worlds cannot modify the GPT or GPC's registers.

**Granule Protection Check 3 (GPC3).** Armv9.6-A introduced GPC3, an upgraded GPC that lets designated memory regions bypass the check by configuring the GPC bypass window register (`GPCBW_EL3`) [23]. Bypass windows are configurable from 1GB to 64GB. Each core has its own `GPCBW_EL3`. A core can directly access its configured bypass windows, while other cores must undergo GPC.

### B. Limitations of Existing Solutions

Intra-process isolation is the leading technique for enforcing the principle of least privilege. Existing mechanisms (see Table II) partition memory into distinct domains within a single address space and use specialized hardware instructions to switch between them. For example, lwC [27] builds contexts by assigning each a separate virtual memory mapping via page tables. Context switches require updating the page-table base, and even with PCID support to avoid global TLB flushes, the transition still traps into privileged mode, incurring non-negligible overhead.

Unlike page-table-based approaches, Intel MPK [13] enables unprivileged domain switching, achieving $< 2\%$ overhead when isolating SSL keys in Nginx [28]. However, MPK supports only 16 domains, which hampers scalability in multi-tenant workloads, prompting new designs that lift this limit. For instance, Donky [29] leverages the 10 reserved bits of the PTE to support up to 1,024 domains but requires substantial hardware modifications. libmpk [30] employs an LRU-based algorithm to evict inactive domains, reusing their domain IDs without hardware changes. However, evicting a domain involves modifying the PTEs of all associated memory pages. As the number of pages or domains grows, eviction costs and rates increase, leading to significant overhead. EPK [31] leverages Intel's Extended Page Table (EPT) to reuse MPK domain IDs. Since the `VMFUNC` instruction used for EPT switching is also non-privileged, domain switching can be efficiently performed in user space. LightZone [32], like lwC, creates separate memory mappings for domains to enhance scalability, theoretically supporting up to 65,536 domains. The key difference is that LightZone runs processes in kernel space to avoid trapping overhead of domain switching. Crucially, however, both EPK and LightZone depend on a trusted OS or hypervisor to manage these isolation structures. This dependency not only inflates the TCB but also renders them insecure under the adversarial kernel threat model addressed by NanoZone.

TABLE II
COMPARISON OF INTRA-PROCESS/ENCLAVE ISOLATION.

| System | Efficiency[1] | Scalability[2] | Security[3] | | Platform |
|---|---|---|---|---|---|
| | | | PA | SA | |
| lwC [27] | Slow | Many | ✗ | ✗ | Portable |
| MPK [13] | Fast | 16 | ✗ | ✗ | Intel |
| Donky [29] | Fast | 1,024 | ✗ | † | RISC-V |
| libmpk [30] | Slow | Many | ✗ | ✗ | Intel |
| EPK [31] | Fast | 7,680 | ✗ | † | Intel |
| LightZone [32] | Fast | 65,536 | ✗ | † | Arm |
| Nested Enclave [11] | Slow | Many | ✓ | † | Intel |
| LightEnclave [12] | Fast | 16 | ✓ | † | Intel |
| HiveTEE [33] | Slow | Many | ✓ | † | Arm |
| **NanoZone** | **Fast**[4] | **Many** | ✓ | ✓ | **Arm** |

[1]**Efficiency** refers to domain-switching performance. We classify efficiency as "Fast" if domain switching can be performed primarily in user space (e.g., in the low hundreds of cycles).
[2]**Scalability** refers to the maximum number of supported domains.
[3]**Security** refers to the ability to resist the PA (Privilege Attack) from the OS or the internal SA (Switch Abuse). † indicates incomplete defense.
[4]Due to NanoZone's hierarchical design, we consider the amortized switching cost here.

While intra-enclave isolation effectively mitigates threats posed by privileged adversaries, it struggles with efficiency and scalability. Nested Enclave [11] incurs substantial overhead during switches between outer and inner enclaves, whereas MPK-based solutions such as LightEnclave [12] inherently restrict the number of isolation domains to 16. The concurrent HiveTEE [33] lifts this limit on Arm CCA by reusing MTE tags across GPTs, but every entry into its domains still traps to the Monitor. On the other hand, domain-switching abuse creates internal attack vectors that are often overlooked. lwC [27] and libmpk [30] ignore these threats entirely. More advanced designs like Donky [29], EPK [31], LightEnclave [12], Nested Enclave [11], LightZone [32], and HiveTEE [33] implement *gate-centric defenses*. By authenticating the caller's identity, execution state, or entry target, they effectively prevent **unauthorized entry** (arbitrary jumps or returns to the trampoline entry). Furthermore, they mitigate **mid-trampoline injection** either through hardware enforcement (Donky, Nested Enclave) or by verifying post-switch execution states, such as random nonces (EPK, LightEnclave) or call-site allowlists (LightZone). However, these defenses remain fundamentally vulnerable to **borrowed-gate attacks** (or confused-deputy attacks). In this scenario, an intra-

process adversary exploits memory corruption to overwrite control-flow data (e.g., return addresses or function pointers) in untrusted code, redirecting execution to a *legitimate* call site that invokes the trampoline. Since the domain switch is triggered from a valid location, gate-centric checks approve the transition, unaware that the control flow leading to it has been hijacked. Addressing this threat requires moving beyond gate verification to enforcing Control-Flow Integrity (CFI) on the path to the switch, as implemented in NANOZONE.

## III. SYSTEM OVERVIEW

Existing intra-process isolation mechanisms face an isolation trilemma, forcing a trade-off between Scalability, Efficiency, and Security. Software-based approaches scale well but suffer from high overhead or weak security, while hardware-assisted ones (like MPK) are efficient but capacity-limited. NANOZONE addresses this by synthesizing complementary hardware capabilities into a hybrid architecture.

**Breaking the Scalability Limit.** To overcome the capacity constraints of single hardware primitives (e.g., POE's 7 domains), NANOZONE adopts a hierarchical composition strategy. It combines user-space POE, kernel-level PIE, and firmware-level GPC3 to instantiate domains across three tiers. By leveraging GPT's ability to map multiple physical address spaces (PAS), we effectively decouple the logical domain count from hardware limits, achieving unlimited scalability.

**Preserving Efficiency via Locality.** Hierarchical isolation introduces latency heterogeneity: switching deep tiers (PAS) is significantly costlier than user-level tiers (POE). We mitigate this by exploiting request locality. NANOZONE's scheduler employs an affinity-based allocation algorithm that groups sequential requests from the same connection into contiguous domains mapped to the same PAS. This maximizes execution residency in the fast L1-Zone (POE), masking the latency of rare privileged transitions.

**Enforcing Security under Adversarial Kernels.** To counter domain-switching abuse without relying on a trusted OS, NANOZONE anchors its root of trust in the minimal root-world Monitor. We enforce Code-Pointer Integrity (CPI) to secure the entire control-flow path leading to the switching trampoline, preventing borrowed-gate attacks. Furthermore, the Monitor offloads critical configuration tasks (e.g., PIE/POE setup) from the OS, ensuring isolation integrity even against a compromised kernel.

**System Architecture.** The architecture of NANOZONE is illustrated in Figure 4. In our design, Arm CCA serves as the hardware substrate rather than the software deployment environment: NANOZONE repurposes CCA's GPT, GPC, and GPC3 primitives for isolation, but protected processes run in the normal world and never enter the realm world or use CVM-based isolation. NANOZONE divides the address spaces of protected processes into developer-defined domains across three tiers: POE domains (L1-Zone), PIE domains (L2-Zone), and GPT PASs (L3-Zone). The security of these domains is enforced by the Monitor in the root world. To further minimize the trusted software stack, NANOZONE adopts a delegation-based design that separates *control* from *ownership*.

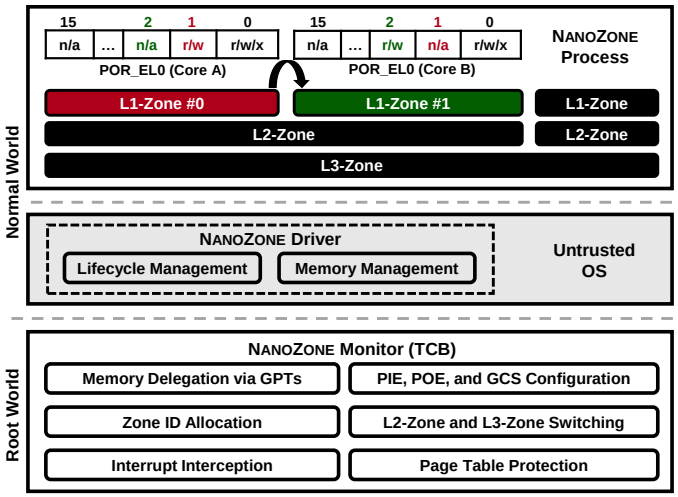



Fig. 4. The architecture of NANOZONE.

Specifically, we introduce a driver to the OS that manages the lifecycle of protected processes and memory allocation. While the OS can create, schedule, and destroy protected processes, it cannot access their data, code, page tables, or register states once the processes are delegated.

During compilation, we modify the LLVM compiler to identify all code pointers (e.g., function pointers, return addresses) for instrumentation protection, combined with post-compilation binary scanning to eliminate unsafe pre-inlined domain-switch or GCS store instructions (§IV-B).

In the setup phase, the OS first loads the instrumented and scanned binaries and then invokes the driver to pre-allocate and map memory for isolation domains, along with their security control structures (e.g., page table and control-flow backups). The driver requests the Monitor to audit the loading procedure and delegates the pre-allocated physical memory to the protected process by configuring the OS's GPT, ensuring that the OS cannot access these memory regions. The Monitor then configures the protected process's GPT to isolate all L3-Zones from intra-process adversaries (§IV-C). Next, the Monitor partitions the L3-Zone into L2-Zones and L1-Zones through PTE configurations (i.e., PIE and POE indexes) and marks the control-flow-backup pages with the GCS attribute (§IV-A). Finally, the Monitor initializes all registers associated with these features.

During runtime, the Monitor assigns isolation domain IDs for the current execution context. The user space facilitates L1-Zone context switching alongside CPI enforcement, which mitigates borrowed-gate attacks, without requiring privileged transitions to the Monitor. However, switching between L2-Zones and L3-Zones requires the Monitor to update the GPC3 and PIE registers (§IV-A). Throughout process execution, the Monitor intercepts each interrupt to enforce memory isolation and protect the register state. Before resuming the process, the Monitor verifies and synchronizes page table updates to prevent unauthorized modifications to PIE, POE, and GCS configurations (§IV-C).

**Threat Model and Assumptions.** NANOZONE's TCB is confined to the native root-world firmware and the security

modules it adds. This firmware is loaded through CCA Secure Boot, so its integrity can be verified with the vendor's signature. We trust the underlying hardware to implement the required features (e.g., RME, PIE/POE, and GCS) correctly and assume a clean toolchain, meaning the application is built on a secure host. The normal-world OS (including our driver) and the software stacks in the realm and secure worlds are treated as untrusted.

NANOZONE protects against both privileged and intra-process adversaries. Privileged attackers may control the OS to remap domain memory, alter domain configurations, tamper with permission registers, inject malicious interrupts, forge domain-switch requests, or exploit DMA devices. NANOZONE also accounts for Iago [34] attacks that return malicious memory mappings. In parallel, intra-process adversaries may exploit memory bugs [6] within the untrusted code to breach domain boundaries or hijack control flow [35], [25], enabling domain-switching abuse (e.g., mid-trampoline injection, unauthorized entry, and borrowed-gate attacks). Finally, NANOZONE enforces mutual isolation so that a compromised domain cannot target other domains or escalate privilege to higher-level software.

NANOZONE excludes scenarios where isolation domains intentionally leak sensitive data or contain internal vulnerabilities. Physical attacks [36], [37], [38], microarchitectural side channels [39], [40], denial-of-service (DoS) attacks, and Iago attacks beyond memory mapping are outside our threat model.

## IV. NANOZONE DESIGN

### A. Isolation Domain Extension

NANOZONE employs a standard code-centric workflow [41], [42]: developers mark sensitive code, allocate domain memory, and insert call gates to thwart memory-safety exploits [6]. Targeting server workloads, it also supports a data-centric model [43] that isolates per-request contexts on demand. Fundamentally, NANOZONE redesigns the core mechanisms of memory configuration and permission switching, enabling unlimited domain scalability while strategically allocating IDs to minimize expensive privileged transitions.

**Extended Domain Configuration.** With POE alone, the 3-bit POE index (value 0 being reserved) limits the system to seven addressable domains. NANOZONE breaks past this cap in two ways: (1) it taps the remaining unused index encodings, and (2) it reuses the same encoding concurrently for separate, non-overlapping physical memory regions.

PTEs carry a 4-bit PIE field for base permissions in addition to the 3-bit POE field for overlay permissions. While most PIE index encodings are already in use (six for common permissions, two for GCS, and four for kernel), we observe that there remain four unused encodings. As shown in Figure 5(a), NANOZONE combines PIE and POE indexes to form the domain ID, expanding the addressable domains to 28. Isolation between these expanded domains is enforced in two cases. (1) Domains within the same L2-Zone. For example, Domain-A (`PI:6, PO:1`) and Domain-B (`PI:6, PO:2`) share the same base permission (index 6) in the PIE

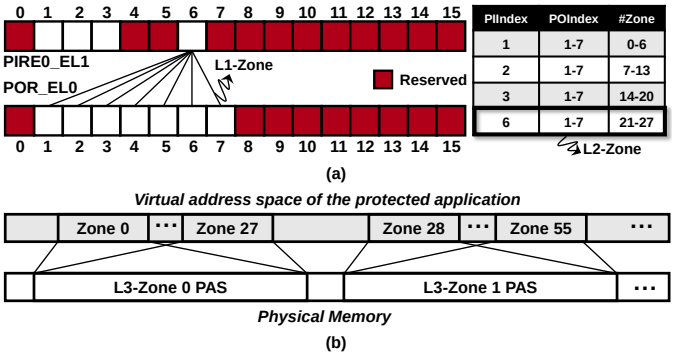



Fig. 5. Structure of the domain extension: (a) shows the extension via PIE, while (b) depicts the extension via multi-PAS.

permission register (`PIRE0_EL1`). However, their overlay permissions differ in the POE permission register (`POR_EL0`). When executing in Domain-A, `overlay perm1` is enabled and `overlay perm2` is forced to 0 (overriding `base perm6`), which blocks any access to Domain-B. (2) Domains across different L2-Zones with a shared POE index. Consider Domain-A (`PI:6, PO:1`) and Domain-C (`PI:3, PO:1`). Both domains enable `overlay perm1`, but Domain-C's base permission (index 3) remains disabled. As a result, even though they share the same POE index, no lateral access is allowed from Domain-A into Domain-C. Notably, PIE indexes with fixed semantics (for example, the default index 4) are never reused, since their base permissions cannot be masked.

The 28 POE-PIE domains still fall short of the scalability needed for multi-tenant servers; therefore, NANOZONE turns to another CCA feature, the GPT, which partitions physical memory into multiple PASs beyond the page-table layer. A simple idea is to map each POE-PIE domain to its own PAS, so a domain is identified by ⟨PAS, PIE, POE⟩; see Figure 5(b). This raises a new problem: the requesting core must gain exclusive access to its target PAS while all other cores are blocked. Existing schemes give every PAS a private access-control table (e.g., EPK [31] maintains extra EPTs), but this per-PAS-per-table model inflates memory and incurs creation latency (both detailed in §VII-B), while being constrained by hardware limits such as 512 EPTs per VM. NANOZONE keeps all PASs inside a single process's GPT and uses the GPC3 bypass-window feature. During setup each PAS is marked `no-access`. When a core needs to access a particular PAS, the Monitor programs that core's `GPCBW_EL3` register to disable the GPC only for the required range; other cores still fail the check. Since the Monitor pre-defines every legal window, attackers cannot open arbitrary ones.

Keeping a single GPT also strengthens isolation under an untrusted OS. Per-PAS-per-GPT schemes grant a core access by rewriting a region's attribute to an accessible state. The correctness of such a scheme then depends on validating, at every rewrite, that the region is not cross-world memory; a single omitted check exposes another world's memory to a protected application. We observed a concrete instance of this risk in the implementation of Shelter [9], which we have reported to its authors. In NANOZONE, by contrast, a protected PAS is

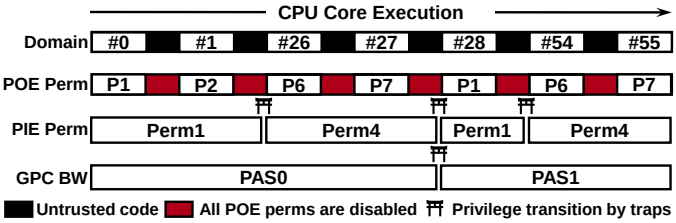



Fig. 6. Intra-process permission switching in the NANOZONE lifecycle.

never rewritten and remains `no-access` throughout; access is determined solely by the Monitor-exclusive `GPCBW_EL3` register.

**Domain Switching.** Like prior works [29], [31], [32], [12], NANOZONE packages its domain-switching code into a dedicated trampoline that orchestrates transitions across the three isolation levels. Permission grants are usually trap-free; each involves enabling the corresponding overlay permission by writing `POR_EL0`, but only transitions that cross L2- or L3-Zone boundaries require a privileged trap. To revoke access, user code simply clears all overlay permissions; even if the PIE base permission or bypass window stays active, the cleared overlay ensures untrusted code cannot reach isolation domains (see Figure 6). For an L2 switch, the Monitor writes `PIRE0_EL1` to disable the old base permission and enable the new one. For domains within an L2-Zone that require special privileges, we set their base permission to full `R/W/X` and use POE overlays for fine-grained access control, avoiding unnecessary traps. Switching between L3-Zones updates `GPCBW_EL3` to point the bypass window at the new PAS.

**Domain ID Allocation Optimization.** L2 and L3 switches trigger privileged traps, so NANOZONE lowers their frequency by redesigning domain allocation. First, it follows a per-connection-per-domain policy, keeping consecutive requests from the same connection in the same domain and limiting them to inexpensive L1 switches. Second, servers like Memcached hand connections to worker threads in round-robin fashion; if domains were allocated in that same order, each thread would own a scattered set spanning many L2 and L3 zones, sharply raising the trap rate. NANOZONE instead grants each thread a block of 28 consecutive domains that all map to the same PAS. Threads still process connections sequentially, but each one lands in a locally contiguous domain. Thus only a small share of requests must perform L2 and L3 switches (see §VII-B for hit-rate details).

### B. Code-Pointer Integrity

The root cause of domain-switching abuse lies in two vectors. (1) pre-inlined switching instructions embedded in the binary, and (2) control-flow hijacking attacks that exploit existing switching logic, whether via returns/jumps to the trampoline or direct mid-trampoline execution of switching instructions. Notably, merely validating that an indirect branch reaches a legitimate landing pad is insufficient in our threat model. An adversary may still corrupt control-flow data in untrusted code and redirect execution to an otherwise legitimate switch site, i.e., a borrowed-gate attack. To counter

```
1 .macro backup_ptr, x1: address of the function to be stored
2                    x2: memory address where the pointer resides
3   ## 1) fetch next idle slot address from PIM[0]
4   mrs     x3, TPIDRRO_EL0          ## x3 ← PIM base
5   ldr     x0, [x3]                 ## x0 ← *PIM[0] (idle slot)
6   ## 2) compute slot index & bounds check
7   sub     x4, x0, x3               ## offset = slot - base
8   lsr     x4, x4, #3               ## index  = offset / 8
9   cmp     x4, #PIM_SIZE
10  b.hs    .Lpim_oob                ## abort if PIM is full
11  ## 3) store function pointer to that slot (GCSSTR)
12  .inst   0xD91F0C01               ## GCSSTR x1, [x0]
13  ## 4) advance idle pointer and write it back to PIM[0]
14  add     x0, x0, #8               ## next_free += 8
15  .inst   0xD91F0C60               ## GCSSTR x0, [x3]
16  ## 5) encode slot index into high-order bits (except bit 55)
17  and     x5, x1, #PTR_MASK        ## clear old high bits
18  bfi     x5, x4, #48, #7          ## index[0:6] → bits 48-54
19  lsr     x4, x4, #7
20  bfi     x5, x4, #56, #8          ## index[7:14] → bits 56-63
21  str     x5, [x2]                 ## publish encoded pointer
22 .endm
23
24 .macro check_call, x1: The function pointer to be called
25  ## 1) duplicate encoded pointer and then strip high bits
26  mov     x2, x1                   ## x2 ← encoded
27  and     x1, x1, #PTR_MASK        ## x1 ← original
28  ## 2) extract 15-bit index and check bounds
29  ubfx    x4, x2, #48, #7          ## low 7 bits  → x4[6:0]
30  ubfx    x7, x2, #56, #8          ## high 8 bits → x7[7:0]
31  lsl     x7, x7, #7
32  orr     x4, x4, x7               ## x4 ← full index
33  cmp     x4, #PIM_SIZE
34  b.hs    .Lpim_oob                ## abort if OOB
35  ## 3) fetch golden copy from PIM and compare integrity
36  mrs     x5,  TPIDRRO_EL0         ## PIM base
37  add     x5,  x5,  x4,  lsl #3    ## slot addr = base + idx*8
38  ldr     x6,  [x5]                ## golden pointer
39  cmp     x1,  x6
40  b.ne    .Lcorrupt                ## abort if mismatch
41 .endm
```

Fig. 7. Instrumentation code for pointer backup and call verification (type-ID handling omitted).

these attacks, NANOZONE first performs binary scanning to eliminate pre-inlined switching instructions (e.g., unauthorized `POR_EL0` updates), ensuring that only the mapped trampoline can change permissions. Second, NANOZONE blocks switching reuse by enforcing CPI on return addresses and function pointers, thereby protecting the entire control-flow path leading to a domain switch.

**Return Address Protection.** NANOZONE employs the off-the-shelf GCS feature to secure return addresses. On each branch-with-link, the CPU pushes the return address to the protected GCS region and, on return, checks it against the value popped from that stack. The enabling and configuration of GCS, such as setting the stack pointer to a securely pre-delegated memory area, are managed by the Monitor.

**Function Pointer Protection.** The core idea behind our function pointer protection is to create a set of legitimate landing pads (indirect jump targets), back them up into the PIM region (a NANOZONE-managed read-only memory area with GCS attributes) and enforce pre-jump integrity checks. Despite conceptual simplicity, it faces three critical challenges: (1) ensuring threads securely acquire their PIM base address, (2) optimizing the efficiency of backup storage and lookup during checks, and (3) preventing reuse attacks that exploit previously backed-up legitimate targets.

We observe that `TPIDRRO_EL0` is a *read-only*, user-accessible per-thread register. The OS commonly uses this zero-latency path to expose values that must remain constant for a thread's lifetime. Following the same practice, NANOZONE stores the PIM base address in `TPIDRRO_EL0`, shielding it from intra-process tampering.

For efficient backups (Figure 7, `backup_ptr`), the first PIM entry holds the address of the next free slot, so user-mode code stores the backup with one non-privileged `gcsstr` and advances the slot address with another. Note that during binary scanning we also strip out any pre-inlined `gcsstr` so that only our instrumentation can invoke it. This does not affect the GCS protection of ordinary applications. The return-address shadow stack is maintained by hardware on `BL` and `RET` with no explicit store, so a normally compiled application contains no `gcsstr`; the instruction performs an explicit GCS write, a capability disabled by default and used only by low-level components such as checkpoint/restore. Stripping it therefore removes an abuse path through which untrusted code could forge PIM entries, without affecting any application's return-address integrity. Lookup is also constant-time (Figure 7, `check_call`): NANOZONE embeds the slot index into the unused high-order bits of the pointer during backup. This index travels with the pointer as it is copied, avoiding extra memory or propagation tracking. To eliminate redundant storage during pointer propagation, NANOZONE checks the high-order bits: if they already contain a valid index, the pointer has been backed up previously and the operation is skipped. NANOZONE retrieves the backup directly from the PIM using the base address and index for comparison with the target. Before utilizing the index, range checks ensure accesses stay within the PIM boundaries, preventing attackers from retrieving forged backups via out-of-bounds indexes (see §VII-B for detailed CPI costs).

Attackers could try to reuse legitimate landing pads. To mitigate this, we introduce pointer-type tags for function pointers. During compilation, we extract each function signature at the LLVM IR level, generate a truncated 64-bit hash (serving as a unique type ID), and embed this ID directly into IR instructions to preserve type context across the compilation pipeline. When backing up function pointers, the ID is stored as metadata alongside the pointer in the PIM. At call sites, both the pointer and its associated ID are retrieved, and the ID is validated against the expected type hash, thereby hindering gadget chaining.

NANOZONE also needs to handle some corner cases. Global function pointers are initialized at compile time, bypassing the runtime backup. We modify the linker script to place all initialized global function pointers into a dedicated segment. During binary loading, this segment is copied into the PIM, and each pointer is stamped with its corresponding PIM index in the high-order bits. Additionally, to address backup omissions from assignments to universal pointers (e.g., `void*`), we insert extra checks to detect whether the source address is a function pointer that has been typecast (i.e., via `bitcast`). If so, the pointer is backed up, and the type ID is derived from the original source type. LLVM IR's strong typing guarantees that any later call through the universal pointer must cast it back to a concrete function-pointer type, and that cast exposes the type we need. Further propagations (even nested ones) need no extra handling because the pointer has already been recorded.

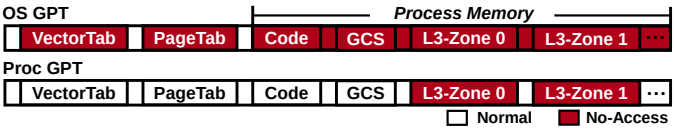



Fig. 8. A dual-GPT design blocking OS access to secure memory.

### C. OS Privilege Reduction

To prevent the OS from breaking the established isolation, NANOZONE restricts its privileges in three ways: (1) it denies the kernel access to secure memory regions, (2) it blocks any kernel changes to sensitive interrupt context, and (3) it forbids unauthorized updates to page-table mappings or permission configurations of POE, PIE, and GCS pages.

**Memory Isolation.** Managing access at the virtual memory layer, such as hooking page tables for the kernel and all other processes to block alias attacks, is both complex and heavyweight. Prior works [9], [8] shift control to the physical memory layer with GPTs, delivering a simpler and more efficient solution. NANOZONE therefore maintains two distinct GPTs: an OS GPT and a proc GPT (see Figure 8). The OS GPT denies the kernel any access to process memory, including domain pages, the GCS and PIM regions, and executable code, and also revokes access to interrupt-vector tables and page tables. Once the driver allocates these regions, the Monitor marks them `no-access` before handing them to the process. The proc GPT, by contrast, leaves everything accessible except the PASs that make up L3-Zones, which stay blocked until a bypass-window switch occurs (§IV-A). Other sensitive areas (e.g., read-exec code) remain protected by ordinary page-table permissions. To ensure both GPTs coexist safely, the Monitor checks that delegated memory never overlaps existing regions and, when the realm world or secure world requests pages, updates the proc GPT to prevent isolation domains from reaching them. For system calls that pass arguments through a process buffer, the Monitor allocates and maintains shared memory to keep the OS from touching process memory directly.

**Interrupt Interception.** To isolate process state from the OS, NANOZONE intercepts all user-to-kernel transitions (see §VII-B for detailed costs). A pre-delegated interrupt vector (see Figure 8) ensures that control first enters EL3 before reaching the kernel. The Monitor swaps the proc GPT for the OS GPT, saves and sanitizes sensitive context (POE/PIE permission registers and GCS/PIM pointer registers), and, just before handing control back, restores this context while re-enabling PIE, POE, and GCS so the OS cannot silently disable them. However, we observe that domain switches occur far more often than interrupts, so forwarding each L2- or L3-Zone transition through a two-stage trap (first to the vector table, then to the Monitor) would be too expensive. Capturing PIE register accesses from EL3 (i.e., via `SCR_EL3`) is also

impractical due to overly coarse control; once enabled, all POE register accesses would likewise trap to the Monitor, undermining our ***efficiency goal*** (§III). Instead, NANOZONE exploits the Armv8.5-A RNG TRAP feature: any user-space read of `RNDR` or `RNDRRS` traps directly to the Monitor. NANOZONE co-opts these registers as lightweight triggers for L2 and L3 switches, entirely bypassing kernel mode. Since RNG trapping also applies to the OS, the Monitor validates each request by checking the current GPT base, blocking domain-switching abuse.

**Page Table Protection.** As noted earlier, NANOZONE configures the OS GPT to deny the OS access to process page tables (see Figure 8). When a page fault occurs, the OS forwards the mapping request to the Monitor, which performs the actual page table updates. The Monitor enforces strict checks to ensure that: (1) newly mapped pages are non-executable, and (2) existing secure memory mappings and their configurations (i.e., PIE/POE indexes in PTEs) are not modified. Rule (1) is deliberately absolute. All code pages, including shared libraries resolved by the dynamic linker, are mapped and audited during the setup phase. After delegation, the Monitor denies any transition of a page to executable. Runtime code loading such as `dlopen` is therefore unsupported by design. This closes the runtime code-injection surface entirely. We note this restriction in §VIII. Additionally, NANOZONE defends against memory-mapping Iago attacks [34], such as malicious `mmap`, by verifying that new mappings do not overlap with existing ones.

## V. IMPLEMENTATION

At the time of writing, no commercial SoC, including the Cortex-X925 [44], supports the Arm extensions our work depends on. Therefore, we implemented: (1) a functional prototype on the Arm Fixed Virtual Platform (FVP) [45], which natively supports these features, to evaluate security properties; and (2) a performance prototype on physical Arm SoCs, where the missing features are emulated, to assess execution overhead.

**Functional Prototype.** NANOZONE instruments source code with LLVM 16.0.0 to secure function pointers. To locate where a function pointer is stored, it iterates over every `StoreInst` and `MemCpyInst`, since the latter often initializes struct fields that are function pointers. With `getPointerElementType`, it checks the pointee type: if the destination is a function pointer, or the source was a function pointer before a `BitCastInst`, it inserts backup-protection code. NANOZONE then analyzes each `CallInst` to identify call sites. When the callee is a runtime `Value` (e.g., from a `LoadInst` or `BitCastInst`), it inserts code that verifies the pointer against its backup. Both backup and verification derive a function-type ID by hashing the type with SHAKE-128, mitigating reuse attacks. While requiring recompilation, this toolchain is essential for intra-process security; it automates CFI insertion and strips unsafe instructions, eliminating manual refactoring. This workflow integrates seamlessly into "Secure DevOps" pipelines, enabling tenants to transparently harden workloads (e.g., Nginx) during the build process.

Since GPC3 supports only contiguous bypass windows, the NANOZONE kernel driver leverages the Contiguous Memory Allocator (CMA) to allocate a contiguous block of PAS for the L3-Zone and remaps it into the process's address space. To support application development, NANOZONE exposes a minimal API including `zone_alloc`/`zone_free`, `zone_begin`/`zone_end`, and memory management via `zone_mmap`/`zone_munmap`. Existing domains can request the Monitor (Trusted Firmware-A 2.9.0) to allocate the pre-mapped memory using these interfaces. Notably, to ensure high performance without burdening the developer, `zone_alloc` implements the affinity-based allocation strategy: it automatically assigns a domain ID that maps to the requesting thread's local PAS, thereby defaulting subsequent accesses to efficient L1-Zone switches. Each interface call is wrapped in an `svc` instruction, which traps into a protected vector table that immediately issues an `smc` to enter the Monitor. Regular interrupt traps follow a similar path to reach the Monitor. Before the exception returns to the process, the Monitor re-enables PIE and POE in `TCR2_EL1`, GCS in `GCSCRE0_EL1`, and RNDR trapping in `SCR_EL3`, ensuring these features stay active in user space. During our development, the latest available Linux kernel (v6.7.0) officially supported only the PIE. Support for POE and GCS had not yet been upstreamed, so we applied patches for these features from Patchwork [46], which were still under review at the time. It is worth noting that starting from v6.13-rc1, these features have been merged into the mainline. To enable these features in the FVP, the required boot parameters must be enabled: `has_rndr_trap`, `has_permission_overlay_s1`, `rme_support_level`, `has_rme_gpc3`, `has_gcs`, and `has_permission_indirection_s1`.

**Performance Prototype.** We built the performance prototype on a Rockchip RK3399 (Armv8) board [47]. Its implementation mirrors the functional prototype, except that the required hardware features are emulated as follows: For L1-Zone switches we emulate accesses to `POR_EL0` with `TPIDR_EL0`. Since both are unprivileged system registers, the emulation latency closely mirrors real hardware behavior. L2-Zone switches repurpose `AFSR0_EL1` as `PIRE0_EL1`, while L3-Zone bypass windows substitute `ACTLR_EL3` for `GPCBW_EL3`. Since the hardware RNG-trap is absent, L2 and L3 transitions are also driven by `svc` and `smc`; actual hardware overhead will therefore be lower than our emulation. We also map `AFSR0_EL3` and `AFSR1_EL3` to `GPCCR_EL3` (GPC control) and `GPTBR_EL3` (GPT base) to measure GPC-setup and GPT-switching costs. PIE domains are emulated by reusing available PTE bits in the direct-memory model, while POE domains use reserved PTE fields. The `gcsstr` instruction is replaced by a standard `str`, and `tlbi paallos`, which normally flushes only cached GPT entries, is conservatively replaced by a full TLB invalidation. Since GPTs are memory-mapped, their creation, maintenance, and teardown overhead is modeled identically in both the functional and performance prototypes.

## VI. Security Evaluation

### A. TCB Analysis

NanoZone's TCB is confined exclusively to the Monitor module residing in the root world (EL3), totaling approximately 3K LoCs. The kernel driver (∼1K LoCs) operates outside the trust boundary and is treated as untrusted, as its compromise does not violate the core isolation guarantees enforced by the Monitor. The compile-time toolchain (∼500 LoCs) is trusted but limited to the build phase, as discussed in the threat model (§III).

**TCB Scope and Functional Breakdown.** The Monitor module's 3K LoCs encompass six well-scoped responsibilities: (1) GPT memory delegation and lifecycle management, (2) PIE/POE/GCS register configuration, (3) domain ID allocation, (4) L2/L3-Zone switching, (5) interrupt interception and context sanitization, and (6) page table protection. Each component has a narrow, verifiable interface, making the TCB small enough to be a realistic target for manual auditing or formal verification. Crucially, by anchoring all security logic in EL3, NanoZone eliminates the need to trust the OS, the RMM, or any realm-world software, which is a critical property given that all of these are treated as adversarial in our threat model (§III).

### B. Theoretical Security Analysis

We analyze security by listing attack vectors from the threat model and detailing how NanoZone mitigates them.

**Untrusted OS**. At startup, the OS might tamper with a program's image or map several binaries onto overlapping pages. The Monitor counters this by validating the load address, rejecting overlapping mappings, and auditing code pages before locking them against further modification. Additionally, verifying the image content against the tenant's intended binary requires a trusted reference measurement, whose integration we discuss in §VIII. To prevent races between concurrent delegation requests, all GPT operations are synchronized with a spinlock.

At runtime, the OS GPT blocks the kernel from touching a protected process's page tables; every mapping request is vetted by the Monitor. The kernel therefore cannot place an isolation-domain or GCS/PIM page outside the assigned L3-Zones or change a page's L1- or L2-Zone assignment by editing its PTE indexes. From EL1, the kernel might still try to (1) tweak POE/PIE permission registers to force an L1/L2 switch, (2) redirect the GCS stack pointer, (3) forge an interrupt-return address, or (4) disable the isolation features. All attempts fail because control always returns to EL3 first, where the Monitor restores the registers and re-enables the features (§IV-C). Any effort to trigger an L2 or L3 switch by touching the RNG TRAP register under the OS GPT is likewise rejected; the Monitor allows access only when execution is under the proc GPT. Iago attacks [34] that manipulate the return values of memory-mapping calls such as `mmap` are thwarted because the Monitor refuses overlapping mappings, and GCS/PIM make a follow-on control-flow hijack even harder. Running in the highest-privilege root world, the Monitor is beyond the kernel's reach: the OS cannot bypass its interrupt interception, GPT memory control, or change EL3 settings (e.g., GPT base or L3-Zone bypass windows). Finally, since GPT entries and bypass windows can reside in the TLB, the Monitor flushes the TLB on each interrupt or L3 switch and configures the `TTBR CnP` bit to prevent cross-core sharing.

Peripheral-side attacks deserve equal caution. Following prior CCA systems [9], [48], [49], [33], we configure the SMMU's GPT to enforce GPC for devices, blocking DMA access to protected memory. Additionally, we flush its translation caches, mirroring CPU-side hardening, to prevent TLB leaks through stale entries or bypass windows.

**Intra-process Adversaries**. Memory isolation for each domain is enforced via a three-tier regime (POE/PIE/GPC3), thwarting even arbitrary out-of-bounds reads/writes by untrusted code (§IV-A). Backed-up return addresses and function pointers stop control-flow hijacks such as ROP [25] or JOP [35], blocking any attempt to abuse domain switches; reuse attacks invoking legitimate targets are limited to same-type targets by type ID validation at call sites (§IV-B).

**Malicious Domains**. A domain runs at EL0 in the normal world, so although it can access its own private memory, it lacks the privilege to access the other worlds or the normal-world kernel. Note that the OS page table can still prevent lateral escalation into other user processes. To address potential collusion with the OS, the Monitor checks that each OS-supplied L3-Zone region does not overlap existing memory and synchronously updates any memory delegations requested by other worlds in the proc GPT (§IV-C).

TABLE III
CVEs used for mitigation analysis.

| CVE-* | Description |
|---|---|
| *Memory-Safety Vulnerabilities* | |
| 2014-0160 | Out-of-bounds read in OpenSSL's TLS heartbeat. |
| 2017-2800 | Off-by-one overwrite in wolfSSL's X.509 parser. |
| 2017-18922 | Heap overflow in LibVNCServer's WebSocket decoder. |
| 2022-24834 | Heap overflow in Redis's cjson library. |
| 2023-3138 | Out-of-bounds write in libX11's `InitExt`. |
| *Control-Flow Hijacking* | |
| 2013-2028 | Return-address overwrite in Nginx's chunked parser. |
| 2015-7805 | Function-pointer overwrite in libsndfile's AIFF parser. |
| 2016-5314 | Function-pointer overwrite in LibTIFF's PixarLogDecode. |
| 2021-44486 | Function-pointer overwrite in YottaDB's `op_write`. |
| 2024-22857 | Function-pointer overwrite in zlog's `zlog_rule_new`. |

### C. CVE Mitigation Analysis

Beyond theoretical analysis, we evaluated NanoZone's practical defenses against known CVEs. On Arm FVP, NanoZone blocks all vulnerabilities in Table III, including five memory-safety flaws and five control-flow hijacks.

Memory bugs typically involve out-of-bounds reads (e.g., CVE-2014-0160) or writes (e.g., CVE-2023-3138) targeting sensitive data. NanoZone places secret pages in dedicated isolation domains, ensuring any unauthorized access triggers a POE/PIE fault. For instance, in Heartbleed (CVE-2014-0160), OpenSSL's `tls1_process_heartbeat` fails to validate payload length, allowing a 64 KB over-read. Under NanoZone, since private keys reside in a protected domain,

this over-read provokes a hardware fault rather than leaking secrets.

Control-flow hijacks exploit corruption to overwrite return addresses (e.g., CVE-2013-2028) or function pointers (e.g., CVE-2021-44486). Consider CVE-2024-22857: a heap overflow in zlog's `zlog_rule_new` allows a crafted configuration to overflow the `record_name` buffer and clobber the adjacent `record_func` pointer. NANOZONE prevents this by validating function pointers against PIM-stored backups on every indirect call, immediately terminating execution if tampering is detected.

We designed two attacks to evaluate NANOZONE's resilience to privileged adversaries, and its defenses blocked both attempts. (1) We mapped the process page tables into kernel space and altered the domain's PTEs, changing the POE and PIE indexes so that the pages would lie in a region with ordinary read/write permissions. The change immediately triggered a GPF. (2) While handling `getpid`, we modified the POE and PIE permission registers in the kernel to grant read access to a domain that should have been private. When control returned to user mode, code outside the domain still could not access its memory because the Monitor had already restored the correct register values.

## VII. PERFORMANCE EVALUATION

### A. Experimental Setup

We evaluate NANOZONE on a Rockchip RK3399 board (Armv8-A) with two Cortex-A72 cores (1.8 GHz) and four Cortex-A53 cores (1.4 GHz), a standard configuration in prior works [9], [8], [48]. To eliminate variability from the SoC's big.LITTLE design, we lock the Cortex-A72 cores at 1.8 GHz and disable the little cores. Note that core pinning is employed strictly for benchmarking stability; NANOZONE naturally supports standard OS scheduling, as the Monitor transparently manages hierarchical isolation states during context switches. Our evaluation covers two vectors:

**Micro-benchmarks.** We first evaluate the efficiency of our bypass-window optimization against per-PAS baselines. Subsequently, we dissect the overheads of domain switching, CPI enforcement, and OS monitoring. Specifically, we measure the latency and hit rate of our three-tier switching logic, quantify CPI costs using SPEC CPU2017 [50], and assess OS privilege de-escalation impact via lmbench [51].

**Real-world Applications.** We validate NANOZONE's practicality by hardening three systems: (1) Nginx (isolating SSL keys), (2) Memcached (partitioning K-V pairs), and (3) a persistent NVM data store.

### B. Evaluation on Micro-benchmarks

**Initialization Optimization.** To quantify the benefits of our bypass-window design during setup, we reimplemented the state-of-the-art per-PAS-per-GPT scheme from CAGE [48] as a baseline. Both approaches construct the same OS GPT and proc GPT (Figure 8), but diverge thereafter. In the baseline, each new L3-Zone requires cloning the proc GPT, marking its PAS as normal (i.e., accessible), and assigning the copy to that zone. Note that the Level 1 (L1) tables in the proc GPT can be reused and each PAS is referenced by a single block descriptor in the Level 0 (L0) table, only the L0 table needs to be duplicated per zone. In contrast, NANOZONE always reuses the original two GPTs; each core simply programs its bypass-window register to open the PAS. Under a 25-zone workload, this saves 100 KB of DRAM compared to the baseline, while the performance overhead of L0-table duplication in the baseline is negligible (0.02%).

TABLE IV
COST OF BASIC NANOZONE OPERATIONS.

| Operation | L1 Switch | L2 Switch | L3 Switch | Ptr. Backup | Ptr. Check | getpid | Hooked getpid |
|---|---|---|---|---|---|---|---|
| Cycles | 74.13 | 6,169.47 | 6,173.36 | 18.02 | 11.07 | 725.36 | 6,533.63 |

TABLE V
AVERAGE DOMAIN-SWITCHING OVERHEAD (CYCLES) AND SWITCHING-LEVEL HIT RATES.

| Domain Num.[1] | 7 | 14 | 28 | 112 | 168 | 224 |
|---|---|---|---|---|---|---|
| Avg. Switch Cost | 86.00 | 198.42 | 255.88 | 300.63 | 303.06 | 303.43 |
| L1-Zone Hit Rate[2] | 100.00% | 98.21% | 97.32% | 96.65% | 96.58% | 96.52% |
| L2-Zone Hit Rate[2] | 0.00% | 1.79% | 2.68% | 2.68% | 2.68% | 2.68% |
| L3-Zone Hit Rate[2] | 0.00% | 0.00% | 0.00% | 0.67% | 0.74% | 0.78% |

[1] Domain Number refers to the number of domains assigned per core.
[2] Hit Rate refers to the proportion of switches whose highest switching level is L1, L2, or L3.

**Domain Switching Cost.** To quantify base overheads, we used the PMU to measure CPU cycles over one million iterations (Table IV). The results highlight a stark contrast: a lightweight L1 switch consumes only 74.13 cycles, whereas privileged L2 and L3 transitions exceed 6,100 cycles. This disparity motivates our locality-aware allocation strategy to maximize inexpensive L1 transitions. We evaluated switching overhead using `twemperf` [52] and varied the number of isolation domains while maintaining a constant request rate. Domain switches were strictly determined based on our allocation strategy. With up to 7 domains, the system maintains a 100% L1 hit rate, incurring just 86 cycles per switch (Table V). As we scale to 28 domains, L2 transitions become necessary but remain infrequent; the L1 hit rate stays high at 98.21%, keeping the average cost to 256 cycles. Even at a massive scale of 224 domains, L3 switches occur only 0.78% of the time. Consequently, the average switching overhead stabilizes at roughly 300 cycles, which is less than half the cost of a standard `getpid` system call. We capped the evaluation at 224 domains because the performance trend stabilizes beyond this point, and this scale already exceeds the empirical evaluation of prior solutions: LightZone [32] evaluated a maximum of 128 domains despite its higher theoretical limit.

**CPI Enforcement Overhead.** Table IV confirms the efficiency of our CPI primitives (§IV-B): both backup and verification operations complete in under 20 cycles, benefiting from purely user-space memory accesses. To assess the impact on high-stress workloads, we utilized SPEC CPU2017 [50]. Due to the board's 4 GB RAM constraint, we restricted our evaluation to the C benchmarks from the SPEC rate2017 suite. We tested three configurations: (1) GCS protection for return addresses, (2) PIM backup for function pointers, and (3) integrated

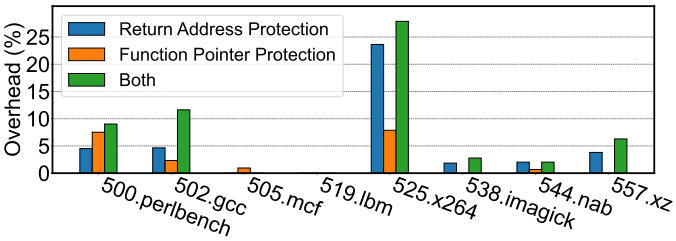



Fig. 9. Performance overhead on the SPEC CPU2017 C benchmarks.

protection for both.

As illustrated in Figure 9, protecting return addresses (GCS) incurs a moderate average overhead of 4.83%. Benchmarks with small code footprints, such as `505.mcf` (∼3K LoCs) and `519.lbm` (∼1K LoCs), exhibit negligible impact. The highest overhead is observed in `525.x264`, attributed to its video encoding workload which involves deeply nested loops and high-frequency function invocations during macroblock processing. Protecting function pointers (PIM) results in a lower average overhead of 2.37% due to their relative infrequency. While benchmarks like `538.imagick` and `557.xz` show minimal impact, `perlbench` experiences a higher 7.52% overhead as its interpreter core relies heavily on function pointers for opcode dispatch. Overall, the integrated protection introduces an average overhead of 7.13%. We also tested Nginx and found that CPI alone reduced throughput by just 1.61%, confirming its practicality in production settings.

**Runtime Monitoring Impact.** NANOZONE intercepts the control flow between a protected process and the OS, introducing additional overhead to kernel services the process relies on. Hooking the trivial `getpid` call adds about 5,808 cycles (Table IV), reflecting only the cost of trap entry/exit, context save-and-restore, and GPT switching that the Monitor performs (§IV-C). More complex syscalls demand extra work from the Monitor, so we used lmbench v3.0-a9 [51] to profile the overall impact. As Table VI shows, monitoring adds less than 10 µs to most operations. The most expensive one is `mmap`, at 36.6 ms; this reflects the Monitor validating and updating page tables for the large region that this benchmark repeatedly maps and unmaps, rather than a fixed per-call cost. To characterize the overhead on the path more relevant to application startup, which is dominated by anonymous mappings, we further profiled an allocation-intensive workload that maps 64 MB of anonymous memory split across $N$ mappings ($N$ from 1 to 8,192) and write-touches every page. On the mapping path, NANOZONE adds about 5.3 µs per call, with the average per-call cost being 2.1 µs natively and 7.4 µs under NANOZONE. On the first-touch path, NANOZONE adds about 0.09 µs per page. Even at the extreme of 8,192 mappings, the total startup time is 74 ms under NANOZONE against 31 ms natively.

TABLE VI
LMBENCH PERFORMANCE OVERHEAD ($\mu$S).

| System | null call | null I/O | stat | open/ close | select TCP | sig inst | sig hndl | fork proc | mmap lat. | page fault |
|---|---|---|---|---|---|---|---|---|---|---|
| **Native** | 0.47 | 0.52 | 1.72 | 2.82 | 6.95 | 0.63 | 2.47 | 1.1K | 13.2K | 0.81 |
| **NANOZONE** | 3.70 | 4.85 | 8.17 | 11.4 | 11.5 | 3.94 | 7.88 | 12.0K | 49.8K | 1.14 |

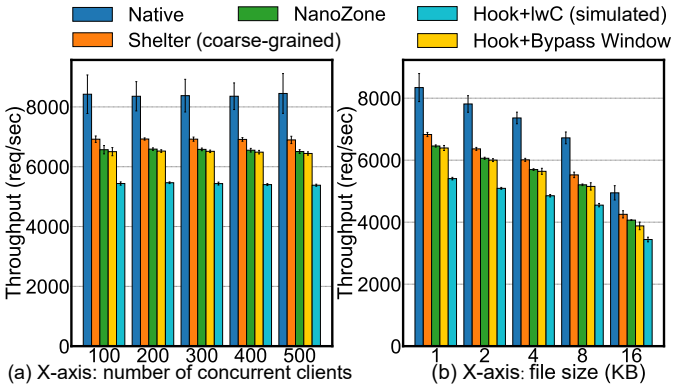



Fig. 10. Comparison of Throughput Overhead on Nginx.

### C. Evaluation on Real-world Applications

**Comparison Systems.** We benchmark against native execution and three alternative schemes: (1) Shelter [9], representing process-level isolation where all requests share a single address space; (2) Bypass-window-only, our baseline that maps each domain to a bypass window and traps to the Monitor on every switch; and (3) lwC [27], adapted from FreeBSD with overheads estimated for privileged emulation [53], [31]. We use these case studies primarily to characterize performance and switching behavior under realistic workloads. We do not claim they provide complete end-to-end security against CIVs [54] without additional interface hardening.

**Case Study 1: Server Application Protection.** We integrated NANOZONE into Nginx v1.26.3 linked with OpenSSL v3.3.4 [55], isolating each client's SSL session key in a separate domain [30] while enforcing strict domain-switch validations. The server was configured with two worker processes and keep-alive enabled on our dual-core board to handle traffic from a remote `ab` [56] client. We evaluated throughput across two dimensions: (1) *concurrency*, varying clients from 100 to 500 (80 concurrent requests, 22 B payload); and (2) *payload size*, ranging from 1 KB to 16 KB (300 clients). Reported results represent the average of 50 repetitions under CPU saturation, where any instrumentation overhead directly degrades throughput.

Figure 10(a) presents the results of the concurrency test. Overall, NANOZONE incurs a 21.84% average throughput overhead. Compared to Shelter [9], which provides only privileged isolation, our finer-grained protection adds just 5.15% overhead, validating the efficiency of our three-tier zone design. Further comparisons reinforce this advantage: relative to the bypass-window-only scheme, NANOZONE achieves 1.00% higher throughput, and it outperforms lwC [27] by 20.93%. These improvements stem from the fact that both of those schemes trap to higher privilege levels on each domain switch. Figure 10(b) shows the file-size test results, which mirror the concurrency findings. Here, the overall average overhead is 21.59%. Against Shelter, performance drops by 5.12%, while

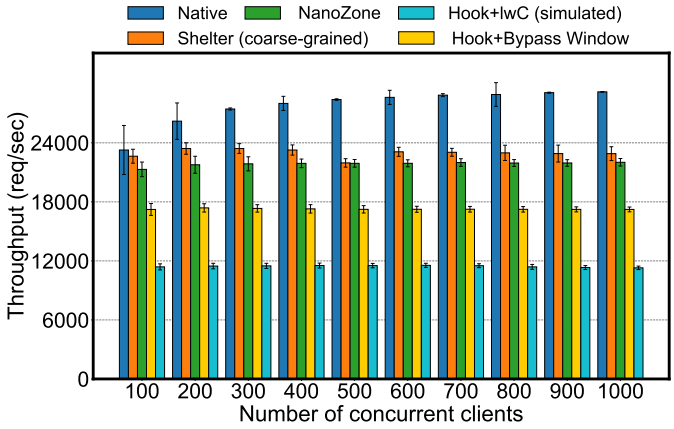



Fig. 11. Comparison of Throughput Overhead on Memcached.

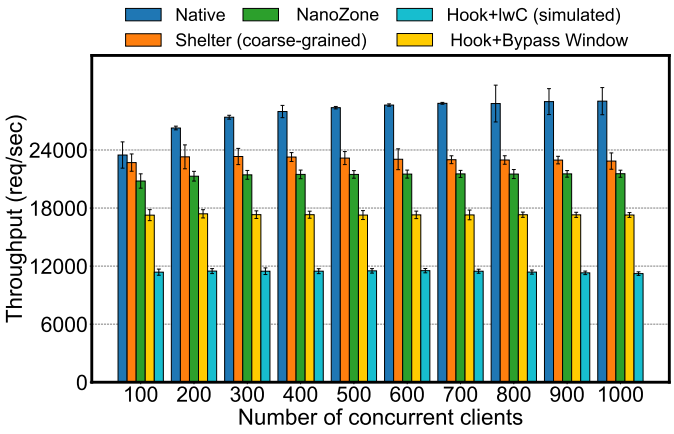



Fig. 12. Throughput on Memcached under round-robin domain allocation.

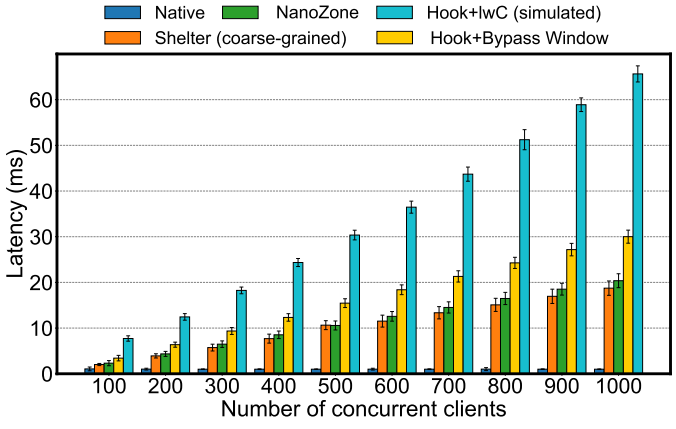



Fig. 13. P95 latency on Memcached across concurrency levels.

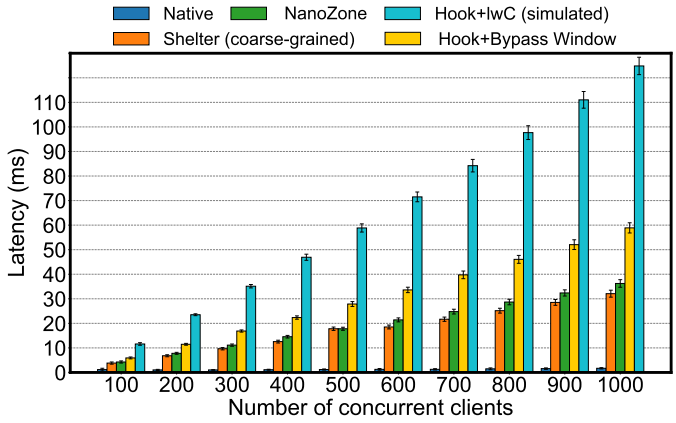



Fig. 14. P99 latency on Memcached across concurrency levels.

throughput improves by 1.76% and 17.70% over the bypass-window-only scheme and lwC, respectively.

**Case Study 2: In-Memory Key-Value Shielding.** We evaluated NANOZONE on Memcached v1.6.31 [57] to quantify the overhead of partitioning a high-performance in-memory key-value store. The server was configured with a 512 MB preallocated heap, where each client's key-value pairs were isolated in a dedicated domain protected by trampoline-based access controls. Running with two worker threads, the server handled traffic generated by a remote `twemperf` [52] client. The workload consisted of 100–1,000 connections, each issuing 30 concurrent requests for 12-byte keys and 64-byte values. We report the average throughput computed from 300 repetitions under full CPU saturation.

Figure 11 illustrates the throughput comparison. NANOZONE incurs a moderate average overhead of 21.06% relative to native execution, reaching about 24.6% under high load. Notably, the majority of this cost is inherent to the baseline privileged isolation (§IV-C): compared to Shelter [9], NANOZONE retains 95.23% of the throughput. In contrast, NANOZONE significantly outperforms alternative intra-process isolation schemes, achieving 26.60% and 90.99% higher throughput than the bypass-window-only baseline and lwC [27], respectively. These gains validate our domain-allocation optimization (§IV-A), which successfully confines the vast majority of operations to user space, thereby avoiding the costly privileged transitions that penalize the other approaches.

To examine NANOZONE under frequent privileged switches, we repeated this experiment with round-robin domain allocation, leaving all other settings unchanged. This allocation scatters each thread's domains across L2- and L3-Zones and lowers the L1 hit rate to 77.29% across 100–1,000 connections. At 1,000 connections, the number of L2 and L3 switches is 6.6× that at 100 connections. Figure 12 illustrates the resulting throughput. Compared to Shelter, the average drop grows only from 4.77% under the default allocation to 7.14%, and is 5.72% at 1,000 connections. NANOZONE still outperforms the bypass-window-only baseline at every concurrency level.

We also measured tail latency, which is a primary concern for server workloads. Figure 13 and Figure 14 report P95 and P99 latency across concurrency levels. Both percentiles grow smoothly and monotonically with load, and NANOZONE's P99 stays at 1.68×–1.84× its P95 throughout, indicating a well-behaved tail without sporadic slowdowns. At 1,000 connections, NANOZONE's P99 is 36.2ms, close to Shelter's 32.1ms and far below the schemes that trap on every switch, whose tails grow much faster (lwC reaches 124.8ms). The absolute values are governed mainly by request queuing under

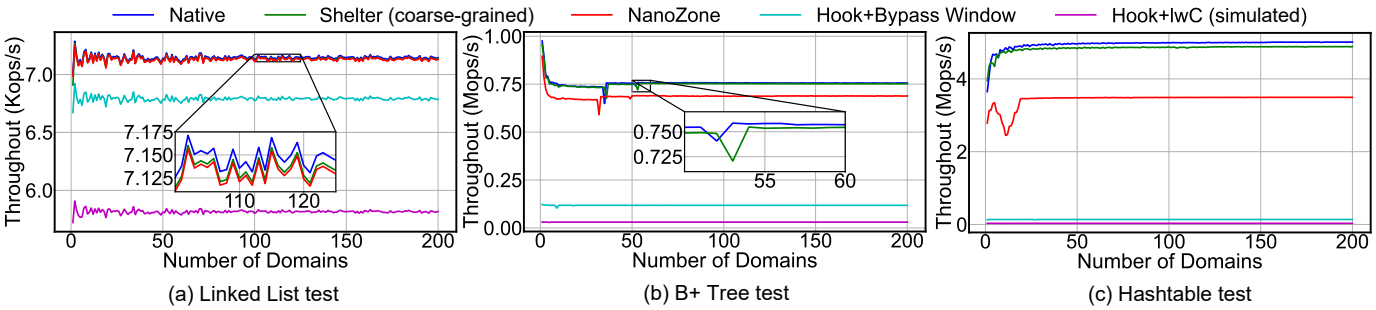



Fig. 15. Data-structure benchmarks: (a) linked list (20 KB per domain); (b) B+ tree (5 MB per domain); (c) hash table (20 KB per domain).

high concurrency, which affects all schemes once throughput saturates; the marginal contribution of NANOZONE's fine-grained isolation over the coarse-grained Shelter baseline stays within about 4.2 ms across the range.

**Case Study 3: NVM Data Isolation.** To mitigate risks when mapping NVM directly into a process, we evaluate intra-process isolation using emulated NVM [58], [59] and three standard data-structure benchmarks [31] (linked list, B+ tree, hash table). We measured throughput by switching domains before and after each random query, varying domain counts from 1 to 200 with over 1,000 repetitions per query.

Figure 15 illustrates that protection overhead is inversely proportional to query latency. NANOZONE incurs negligible overhead (0.21%) on the compute-intensive linked list, rising to 9.09% for the B+ tree and 30.68% for the fast hash table, where domain switching dominates execution time. However, NANOZONE's optimized switching significantly outperforms baselines that rely on privileged traps. In the linked-list benchmark, the bypass-window-only scheme and lwC [27] incur 5.01% and 22.61% higher overhead, respectively. These gaps widen dramatically for memory-bound workloads: on the B+ tree, their overheads are 4.82× and 21.55× greater than NANOZONE's, reaching 24.22× and 108.92× for the hash table. In contrast, Shelter [9] maintains near-native performance (max 2.30% drop) due to low system-call reliance. Compared to Shelter, NANOZONE's fine-grained isolation cost is minimal for slow queries (0.05% on linked list) but noticeable for faster ones, reaching 8.59% on the B+ tree and 29.04% on the hash table.

## VIII. DISCUSSION & LIMITATION

**Security Boundaries.** NANOZONE's type-ID mechanism derives a per-function-signature hash via SHAKE-128, enabling fine-grained validation at each indirect call site and substantially shrinking the set of admissible targets compared to coarse-grained mechanisms such as Arm BTI [24]. Same-type reuse remains a residual limitation: an attacker may still redirect execution to a legitimate target with a matching type signature, an inherent constraint of type-based forward-edge CFI. Additionally, microarchitectural side-channel and denial-of-service attacks remain outside NANOZONE's threat model (§III), as in comparable systems, but the tiered design warrants a brief discussion of both. For timing side channels, an L2 and an L3 switch have almost identical cost (Table IV), so execution timing separates only a fast L1 switch from an expensive L2/L3 one. This reveals at most whether a switch left the current L2-Zone, not which of the 28 domains in a zone was accessed, a signal far coarser than domain identity. For denial of service, a switch at any tier writes only registers on the executing core, and the global lock is taken when an L3-Zone is allocated or destroyed, never on the switching path. A client issuing frequent expensive switches therefore exercises only the lock-free switching path and cannot block other cores by holding a shared lock. Lock contention is confined to zone allocation and destruction, whose rate is set by connection establishment rather than by request processing. A client also cannot select its own tier, since placement is decided by the Monitor's affinity-based allocation and OS scheduling rather than by the client. Moreover, NANOZONE audits the load path through the Monitor, which enforces layout and instruction-level invariants before delegation (§VI-B); verifying the image content against the tenant's intended binary additionally requires a trusted reference measurement. Full measurement and remote attestation form an orthogonal direction. The Monitor's load-path intervention provides a natural attachment point for such a mechanism, which we leave to future work.

**Deployment Boundaries.** Four practical constraints limit immediate deployment. First, NANOZONE depends on POE/PIE, GCS, and GPC3, which are not yet available on any commercial SoC. Our performance prototype emulates these features on a Rockchip RK3399 board. Native hardware will eliminate the two-stage trap path that dominates our switching estimates and reduce reported overheads accordingly. On transitional hardware that lacks GCS or GPC3, NANOZONE remains deployable, but with weaker security guarantees. GPC3 can be emulated using a per-PAS-per-GPT scheme, at the cost of losing the fail-closed property provided by our single-GPT design (§IV-A). Similarly, GCS can be replaced with Armv8.3 Pointer Authentication [60], [61], though this leaves the backward edge vulnerable to pointer-reuse attacks. Second, NANOZONE's L3-Zone design requires physically contiguous memory of 1 GB to 64 GB via the Linux CMA. Even with early boot reservation, large allocations may still fail if pinned or unmovable pages block compaction, or if internal fragmentation leaves no sufficiently large free extent. In that case, the application can be split into multiple processes that each use at most seven domains, so that no L3-Zone is needed. Each switch between these processes, however, passes through the Monitor and costs at least as much as an L2/L3 switch (Table IV). The number of processes also

grows with the number of principals, and data shared across them must be split or shared through the Monitor. Third, NANOZONE requires recompilation with a modified LLVM toolchain to instrument function pointers and strip unsafe instructions, which precludes protection of pre-built binaries and introduces a dependency on a trusted build environment. Fourth, code pages are fixed at launch. After delegation, the Monitor denies any transition of a page to executable (§IV-C). Runtime code loading such as `dlopen` or JIT compilation is therefore unsupported. This is a deliberate trade-off that closes the runtime code-injection surface. Our evaluated server workloads load all modules at startup and are unaffected.

**Performance Boundaries.** NANOZONE's low overhead relies on request locality keeping the majority of domain switches in the user-space L1 tier. For workloads whose access pattern lacks such locality, where consecutive operations scatter across many domains, the share of privileged L2 and L3 switches rises and the average switching cost increases accordingly (see §VII-C for a round-robin stress test); the locality-aware allocation (§IV-A) is therefore most effective when requests exhibit the temporal locality typical of per-connection server workloads. Furthermore, NANOZONE intercepts memory-mapping operations, so workloads dominated by repeated mapping and unmapping of large shared regions incur higher Monitor overhead (§VII-B). It is therefore best suited to server applications whose bottleneck lies in compute or network I/O. Allocation-intensive startup nonetheless remains bounded (§VII-B), and pre-allocating memory pools via `zone_alloc` further reduces mapping frequency.

## IX. Related Work

**MPK-based Mechanisms.** Intel MPK has become the standard primitive for fast intra-process isolation on x86, and has been used to secure language runtimes [62], [63], compartmentalize OS components [64], [65], and optimize process abstractions [66], [53]. A separate line of work scales MPK beyond its domain limit, by modifying hardware, evicting and reusing domain IDs, or reusing IDs through virtualization-level page tables (§II-B). NANOZONE differs in two respects. First, it reuses POE/PIE indexes across distinct physical address spaces through GPT, rather than through extra tables or ID eviction. Second, unlike schemes that depend on a trusted OS or hypervisor to manage these structures, it anchors enforcement in the root-world Monitor and stays secure under an adversarial kernel. More broadly, these advances rely on x86-specific hardware, whereas NANOZONE delivers an equally lightweight facility on Arm.

**Hardware-assisted Control-flow Integrity.** While x86 leverages Intel CET for native shadow stacks [67], Arm architectures long lacked an equivalent hardware mechanism. Prior works approximated this functionality using Pointer Authentication (e.g., PARTS [60], PACTight [61]) or PAN extensions (e.g., PANIC [68]), but these approaches remain susceptible to pointer reuse or incur substantial complexity. With the advent of Armv9, NANOZONE is the first to leverage the native Guarded Control Stack (GCS) to enforce strict, bidirectional control-flow integrity.

**OS Security Monitors.** A range of systems interpose a trusted layer at a higher privilege level to protect applications from an untrusted OS. Hypervisor-based designs such as Overshadow [69] and InkTag [70] shield an application's memory from the OS. However, they protect the application as a single unit, leaving its interior exposed, and add a full hypervisor to the TCB. Others reduce the trusted stack using hardware modifications [71] or platform-specific mechanisms such as AMD VMPLs [14], [15]. For designs built on Arm CCA primitives, placing the monitor at Realm EL2 is suboptimal because it pulls the complex RMM into the TCB. NANOZONE departs from these in two ways. First, it enforces a fine-grained boundary within the process, closing the intra-process gap these systems leave open. Second, it anchors security in a minimal root-world monitor at EL3, rather than trusting a hypervisor or the RMM. Unlike RContainer [8], which offloads some tasks to a mini-OS to reduce root-world usage, NANOZONE consolidates critical enforcement in the monitor to maintain a single, minimal root of trust.

**CCA-based Isolation.** Shelter [9] provides process-level CCA isolation, but treats each application as a single protection unit. As a result, it provides no intra-process separation and leaves the application interior unprotected. NANOZONE adds this interior boundary at a small marginal cost, only 5.2% over Shelter (§VII-C). Other systems apply CCA primitives in different directions. CAGE [48] and Portal [49] use them for confidential GPU computing and secure device access. Closest to our work is HiveTEE [33], a concurrent system that, like NANOZONE, reuses GPTs to scale domains beyond a fixed hardware limit, virtualizing MTE tags where NANOZONE virtualizes POE/PIE indexes. The key difference is the switching path: every HiveTEE domain entry is mediated by the Monitor, whereas NANOZONE performs most switches in user space (Table V). Because these user-space gates grant permissions to the calling thread, NANOZONE must enforce CPI on the path leading to them (§IV-B) and shield the entire process from the OS. HiveTEE validates entries but leaves CFI to orthogonal work, and shields only annotated functions, which keeps its overhead below 3% but leaves the rest of the application untrusted. HiveTEE's MTE tags, in turn, offer 16-byte granularity but must be forced on every memory access through instrumentation.

## X. Conclusion

Arm CCA isolates software only at VM granularity, leaving applications exposed to a compromised OS and to vulnerabilities in their own code. NANOZONE addresses both by protecting normal-world processes from a minimal root-world monitor and partitioning them into fine-grained domains. It combines POE, PIE, and GPT into a three-tier hierarchy, keeps most domain switches in user space through locality-aware allocation, and protects the switch paths with CPI. Across our server case studies, this fine-grained protection costs about 5% over process-level isolation.

## References

[1] AMD, "AMD secure encrypted virtualization (SEV)," 2023, https://www.amd.com/en/developer/sev.html.

[2] Intel, "Intel trust domain extensions (Intel TDX)," 2023, https://www.intel.com/content/www/us/en/developer/articles/technical/intel-trust-domain-extensions.html.

[3] Arm, "Confidential compute architecture," 2023, https://www.arm.com/architecture/security-features/arm-confidential-compute-architecture.

[4] K. Zeng, Z. Lin, K. Lu, X. Xing, R. Wang, A. Doupé, Y. Shoshitaishvili, and T. Bao, "RetSpill: Igniting user-controlled data to burn away Linux kernel protections," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 3093–3107.

[5] Z. Lin, Y. Wu, and X. Xing, "DirtyCred: Escalating privilege in Linux kernel," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 1963–1976.

[6] Z. Durumeric, F. Li, J. Kasten, J. Amann, J. Beekman, M. Payer, N. Weaver, D. Adrian, V. Paxson, M. Bailey *et al.*, "The matter of Heartbleed," in *Proceedings of the 2014 conference on internet measurement conference*, 2014, pp. 475–488.

[7] D. Kuvaiskii, D. Stavrakakis, K. Qin, C. Xing, P. Bhatotia, and M. Vij, "Gramine-TDX: A lightweight OS kernel for confidential VMs," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 4598–4612.

[8] Q. Zhou, W. Cao, X. Jia, P. Liu, S. Zhang, J. Chen, S. Xu, and Z. Song, "RContainer: A secure container architecture through extending ARM CCA hardware primitives," in *NDSS*, 2025.

[9] Y. Zhang, Y. Hu, Z. Ning, F. Zhang, X. Luo, H. Huang, S. Yan, and Z. He, "SHELTER: Extending Arm CCA with isolation in user space," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 6257–6274.

[10] Intel, "Intel® software guard extensions (Intel® SGX)," 2023, https://www.intel.com/content/www/us/en/products/details/processors/xeon/features/software-guard-extensions.html.

[11] J. Park, N. Kang, T. Kim, Y. Kwon, and J. Huh, "Nested enclave: Supporting fine-grained hierarchical isolation with SGX," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 776–789.

[12] J. Gu, B. Zhu, M. Li, W. Li, Y. Xia, and H. Chen, "A Hardware-Software co-design for efficient Intra-Enclave isolation," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3129–3145.

[13] Intel, "Intel® 64 and IA-32 architectures software developer manuals," 2024, https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html.

[14] W. Wang, L. Song, B. Mei, S. Liu, S. Zhao, S. Yan, X. Wang, D. Meng, and R. Hou, "The road to trust: Building enclaves within confidential VMs," in *NDSS*, 2025.

[15] A. Ahmad, B. Ou, C. Liu, X. Zhang, and P. Fonseca, "Veil: A protected services framework for confidential virtual machines," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, 2023, pp. 378–393.

[16] V. A. Sartakov, L. Vilanova, D. Eyers, T. Shinagawa, and P. Pietzuch, "CAP-VMs: Capability-Based isolation and sharing in the cloud," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 597–612.

[17] R. N. Watson, J. Woodruff, P. G. Neumann, S. W. Moore, J. Anderson, D. Chisnall, N. Dave, B. Davis, K. Gudka, B. Laurie *et al.*, "CHERI: A hybrid capability-system architecture for scalable software compartmentalization," in *2015 IEEE Symposium on Security and Privacy*. IEEE, 2015, pp. 20–37.

[18] Y. Chen, S. Reymondjohnson, Z. Sun, and L. Lu, "Shreds: Fine-grained execution units with private memory," in *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 56–71.

[19] Y. Zhou, X. Wang, Y. Chen, and Z. Wang, "ARMlock: Hardware-based fault isolation for ARM," in *Proceedings of the 2014 ACM SIGSAC conference on computer and communications security*, 2014, pp. 558–569.

[20] Arm, "Permission indirection and permission overlay extensions," 2024, https://developer.arm.com/documentation/102376/0200/Permission-indirection-and-permission-overlay-extensions.

[21] S. Liu, Z. Jiang, J. Wang, W. Zhou, K. Sun, Z. Chen, and Y. Xie, "More granular, less trust: Enforcing intra-process isolation with Arm CCA in an untrusted management environment," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 12 507–12 522, 2025.

[22] Arm, "Learn the architecture - Realm management extension guide," 2023, https://developer.arm.com/documentation/den0126/latest.

[23] ——, "Granule protection check bypass window register (EL3)," 2024, https://developer.arm.com/documentation/ddi0601/2024-12/AArch64-Registers/GPCBW-EL3--Granule-Protection-Check-Bypass-Window-Register--EL3-.

[24] ——, "Arm® architecture reference manual for A-profile architecture," 2024, https://developer.arm.com/documentation/ddi0487/latest/.

[25] H. Shacham, "The geometry of innocent flesh on the bone: return-into-libc without function calls (on the x86)," in *Proceedings of the 14th ACM conference on Computer and communications security*, 2007, pp. 552–561.

[26] Intel, "Complex shadow-stack updates (Intel control-flow enforcement technology)," 2023, https://www.intel.com/content/www/us/en/content-details/785687/complex-shadow-stack-updates-intel-control-flow-enforcement-technology.html.

[27] J. Litton, A. Vahldiek-Oberwagner, E. Elnikety, D. Garg, B. Bhattacharjee, and P. Druschel, "Light-Weight contexts: An OS abstraction for safety and performance," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 2016, pp. 49–64.

[28] A. Voulimeneas, J. Vinck, R. Mechelinck, and S. Volckaert, "You shall not (by)pass!: practical, secure, and fast PKU-based sandboxing," in *Proceedings of the Seventeenth European Conference on Computer Systems*, 2022, pp. 266–282.

[29] D. Schrammel, S. Weiser, S. Steinegger, M. Schwarzl, M. Schwarz, S. Mangard, and D. Gruss, "Donky: Domain keys – efficient In-Process isolation for RISC-V and x86," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 1677–1694.

[30] S. Park, S. Lee, W. Xu, H. Moon, and T. Kim, "libmpk: Software abstraction for Intel memory protection keys (Intel MPK)," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 2019, pp. 241–254.

[31] J. Gu, H. Li, W. Li, Y. Xia, and H. Chen, "EPK: Scalable and efficient memory protection keys," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, 2022, pp. 609–624.

[32] Z. Yuan, S. Hong, R. Guo, R. Chang, M. Gao, W. Shen, and Y. Zhou, "LightZone: Lightweight hardware-assisted in-process isolation for ARM64," in *Proceedings of the 25th International Middleware Conference*, 2024, pp. 467–480.

[33] H. Huang and F. Zhang, "HiveTEE: Scalable and fine-grained isolated domains with RME and MTE co-assisted," *IEEE Transactions on Information Forensics and Security*, vol. 21, pp. 2300–2312, 2026.

[34] S. Checkoway and H. Shacham, "Iago attacks: why the system call API is a bad untrusted RPC interface," *ACM SIGARCH Computer Architecture News*, vol. 41, no. 1, pp. 253–264, 2013.

[35] T. Bletsch, X. Jiang, V. W. Freeh, and Z. Liang, "Jump-oriented programming: a new class of code-reuse attack," in *Proceedings of the 6th ACM symposium on information, computer and communications security*, 2011, pp. 30–40.

[36] D. Lee, D. Jung, I. T. Fang, C.-C. Tsai, and R. A. Popa, "An Off-Chip attack on hardware enclaves via the memory bus," in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 487–504.

[37] S. F. Yitbarek, M. T. Aga, R. Das, and T. Austin, "Cold boot attacks are still hot: Security analysis of memory scramblers in modern processors," in *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2017, pp. 313–324.

[38] Y. Kim, R. Daly, J. Kim, C. Fallin, J. H. Lee, D. Lee, C. Wilkerson, K. Lai, and O. Mutlu, "Flipping bits in memory without accessing them: An experimental study of DRAM disturbance errors," *ACM SIGARCH Computer Architecture News*, vol. 42, no. 3, pp. 361–372, 2014.

[39] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, J. Horn, S. Mangard, P. Kocher, D. Genkin, Y. Yarom, M. Hamburg, and R. Strackx, "Meltdown: reading kernel memory from user space," *Communications of the ACM*, vol. 63, no. 6, pp. 46–56, 2020.

[40] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher *et al.*, "Spectre attacks: exploiting speculative execution," *Communications of the ACM*, vol. 63, no. 7, pp. 93–101, 2020.

[41] Y. Liu, T. Zhou, K. Chen, H. Chen, and Y. Xia, "Thwarting memory disclosure with efficient hypervisor-enforced intra-domain isolation," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015, pp. 1607–1619.

[42] K. Dinh Duy, K. Cho, T. Noh, and H. Lee, "Capacity: Cryptographically-enforced in-process capabilities for modern ARM architectures," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 874–888.

[43] H. Lefeuvre, N. Dautenhahn, D. Chisnall, and P. Olivier, "SoK: Software compartmentalization," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 3107–3126.

[44] Arm, "Arm Cortex-X925 core technical reference manual," 2024, https://developer.arm.com/documentation/102807/latest/.

[45] ——, "Fixed virtual platforms," 2023, https://support.arm.com/tools-and-software/fixed-virtual-platforms.

[46] Linux Foundation, "Patchwork - the Linux kernel archives," 2024, https://patchwork.kernel.org/.
[47] Radxa, "ROCK 4B," 2024, https://radxa.com/products/rock4/4b/.
[48] C. Wang, F. Zhang, Y. Deng, K. Leach, J. Cao, Z. Ning, S. Yan, and Z. He, "CAGE: Complementing Arm CCA with GPU extensions," in *Proceedings of the 31st Annual Network and Distributed System Security Symposium*, 2024.
[49] F. Sang, J. Lee, X. Zhang, and T. Kim, "Portal: Fast and secure device access with Arm CCA for modern Arm mobile system-on-chips (SoCs)," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 4099–4116.
[50] Standard Performance Evaluation Corporation, "SPEC CPU® 2017 benchmark," 2025, https://www.spec.org/cpu2017/.
[51] L. McVoy and C. Staelin, "LMbench - tools for performance analysis," 2025, https://lmbench.sourceforge.net/.
[52] Twitter, "A tool for measuring memcached server performance," 2024, https://github.com/twitter-archive/twemperf/.
[53] A. Vahldiek-Oberwagner, E. Elnikety, N. O. Duarte, M. Sammler, P. Druschel, and D. Garg, "ERIM: Secure, efficient in-process isolation with protection keys (MPK)," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 1221–1238.
[54] H. Lefeuvre, V.-A. Bădoiu, Y. Chen, F. Huici, N. Dautenhahn, and P. Olivier, "Assessing the impact of interface vulnerabilities in compartmentalized software," in *Network and Distributed System Security Symposium (NDSS)*, 2023.
[55] OpenSSL Foundation, "OpenSSL library," 2024, https://www.openssl.org/.
[56] Apache Software Foundation, "ab - Apache HTTP server benchmarking tool," 2024, https://httpd.apache.org/docs/current/programs/ab.html.
[57] A. Vorobey, B. Fitzpatrick, and A. D. Kasindorf, "memcached - a distributed memory object caching system," 2024, https://memcached.org/.
[58] Y. Xu, Y. Solihin, and X. Shen, "MERR: Improving security of persistent memory objects via efficient memory exposure reduction and randomization," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 987–1000.
[59] Y. Xu, C. Ye, Y. Solihin, and X. Shen, "Hardware-based domain virtualization for intra-process isolation of persistent memory objects," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 680–692.
[60] H. Liljestrand, T. Nyman, K. Wang, C. C. Perez, J.-E. Ekberg, and N. Asokan, "PAC it up: Towards pointer integrity using ARM pointer authentication," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 177–194.
[61] M. Ismail, A. Quach, C. Jelesnianski, Y. Jang, and C. Min, "Tightly seal your sensitive pointers with PACTight," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3717–3734.
[62] P. Kirth, M. Dickerson, S. Crane, P. Larsen, A. Dabrowski, D. Gens, Y. Na, S. Volckaert, and M. Franz, "PKRU-safe: automatically locking down the heap between safe and unsafe languages," in *Proceedings of the Seventeenth European Conference on Computer Systems*, 2022, pp. 132–148.
[63] H. Lei, Z. Zhang, S. Zhang, P. Jiang, Z. Zhong, N. He, D. Li, Y. Guo, and X. Chen, "Put your memory in order: Efficient domain-based memory isolation for WASM applications," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 904–918.
[64] H. Lefeuvre, V.-A. Bădoiu, A. Jung, S. L. Teodorescu, S. Rauch, F. Huici, C. Raiciu, and P. Olivier, "FlexOS: towards flexible OS isolation," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 467–482.
[65] H. Lu, S. Wang, Y. Wu, W. He, and F. Zhang, "MOAT: Towards safe BPF kernel extension," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 1153–1170.
[66] J. Lin, Y. Chen, S. Gao, and Y. Lu, "Fast core scheduling with userspace process abstraction," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, 2024, pp. 280–295.
[67] L. Maar, P. Nasahl, and S. Mangard, "Beyond the edges of kernel control-flow hijacking protection with HEK-CFI," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, 2024, pp. 1214–1230.
[68] J. Xu, M. Xie, C. Wu, Y. Zhang, Q. Li, X. Huang, Y. Lai, Y. Kang, W. Wang, Q. Wei *et al.*, "PANIC: PAN-assisted intra-process memory isolation on ARM," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 919–933.
[69] X. Chen, T. Garfinkel, E. C. Lewis, P. Subrahmanyam, C. A. Waldspurger, D. Boneh, J. Dwoskin, and D. R. Ports, "Overshadow: a virtualization-based approach to retrofitting protection in commodity operating systems," *ACM SIGOPS Operating Systems Review*, vol. 42, no. 2, pp. 2–13, 2008.
[70] O. S. Hofmann, S. Kim, A. M. Dunn, M. Z. Lee, and E. Witchel, "InkTag: secure applications on an untrusted operating system," in *Proceedings of the eighteenth international conference on Architectural support for programming languages and operating systems*, 2013, pp. 265–278.
[71] Y. Song, B. Woo, Y. Han, and B. B. Kang, "Interstellar: Fully partitioned and efficient security monitoring hardware near a processor core for protecting systems against attacks on privileged software," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 198–212.



**Shiqi Liu** received the B.S. degree in Cyber Science and Engineering from the University of International Relations, Beijing, China, in 2022, and the M.S. degree in Cyber Science and Engineering from Huazhong University of Science and Technology, Wuhan, China, in 2025. His research interests include hardware–software co-design and verification.



**Yongpeng Gao** received his B.S. and M.S. degrees from the School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan, China, in 2023 and 2026, respectively. His research interests include system security and software security.



**Mingyang Zhang** received his B.S. degree in the School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan, China, in 2024. He is currently working toward an M.S. degree at the same institution. His research interests include system security and software security.



**Jie Wang** received his Ph.D. degree from the University of Chinese Academy of Sciences (UCAS) in 2021. He was a visiting scholar at George Mason University and is currently an Associate Professor at the School of Cyber Science and Engineering, Huazhong University of Science and Technology. His research interests include system security, trusted computing, and trusted AI accelerators.